

Interacción Completa De IA

000551191

Samuel Parra Cadavid

1. Infraestructura local con Docker

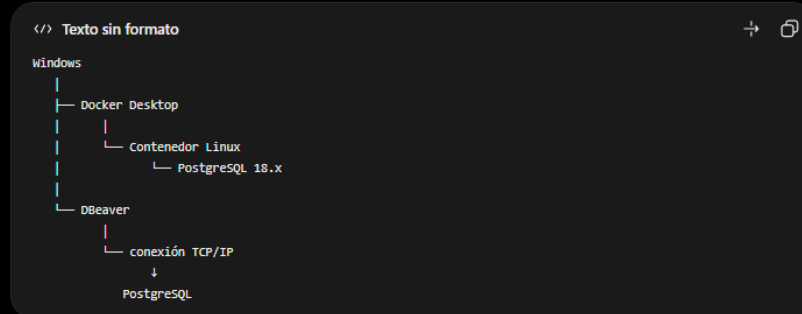
Se definió una infraestructura local con Docker y PostgreSQL 18 para la Etapa 1, conectada posteriormente desde DBeaver.

listo podemos proceder con etapa 1, bro hagámoslo con docker en un contenedor, y esto lo conectamos a dbeaver, entonces toca instalar la imagen de PG en el docker, esta es la infraestructura que te propongo

Sí bro, esa infraestructura está **perfectamente alineada con lo que pide la Etapa 1**.

El examen permite explícitamente una **instalación local mediante contenedores con una distribución Linux de preferencia**, y PostgreSQL 18.x está dentro de los motores aceptados. Además, después debemos evidenciar la conectividad desde un IDE como DBeaver. [tadb_202620_examen02_performanc...](#)

Entonces para tu **Etapa 1** dejamos definida esta arquitectura:



Y como esta vez **no estamos buscando la bonificación de nube**, no hace falta AWS ni crear infraestructura nueva en cloud. Con Docker local cumplimos perfectamente la etapa. [tadb_202620_examen02_performanc...](#)

Etapa 1 — Paso 1: crear PostgreSQL en Docker

Primero comprobemos que Docker está funcionando:

```
</> powershell
docker --version
```

2. Inicio de la Etapa 2

Se identificó que el primer paso debía ser inspeccionar los cuatro archivos antes de diseñar el modelo relacional.

listo empecemos con la etapa 2, El examen dice que esta etapa consiste en tomar los datos suministrados, diseñar e implementar un modelo que permita resolver las consultas posteriores, y de aquí deben salir el diagrama relacional y el script de implementación. deberíamos primero inspeccionar los 4 archivos



Sí, bro. Ese debería ser el primer paso de la Etapa 2.

Antes de diseñar tablas o escribir SQL, tenemos que inspeccionar los **4 archivos/lotes** y responder cosas concretas:

- qué columnas trae cada uno,
- si tienen exactamente la misma estructura,
- cuántas filas tiene cada archivo,
- qué campos parecen identificar organización, brecha, usuario y tipo de dato,
- qué valores se repiten,
- qué dependencias funcionales se ven,
- si hay nulos o inconsistencias,
- y cómo se puede separar la sábana en las **5 tablas esperadas**.

El examen ya nos adelanta que los 40.000 registros vienen en 4 lotes y que detrás hay 60 organizaciones, 800 brechas, 7.887 usuarios, 10 tipos de dato y una tabla puente.  `tadb_202620_examen02_performanc...` Pero **no deberíamos asumir nombres de columnas ni claves todavía**; eso lo sacamos de los archivos reales.

3. Inspección de los cuatro CSV

Se compararon los lotes para identificar columnas, tipos, filas, nulos, duplicados y dependencias funcionales antes de normalizar.

Inspecciona los 4 lotes CSV del examen y compáralos entre sí. Identifica columnas, tipos de datos aparentes, cantidad de filas, nulos, duplicados y estructura común. Detecta posibles claves y dependencias funcionales entre organización, brecha, usuario y tipo de dato. No diseñes aún el modelo final: primero entrega únicamente el diagnóstico de los datos.

Listo bro. Hice solo el diagnóstico, sin diseñar todavía el modelo final.

Los 4 lotes son estructuralmente idénticos: cada uno tiene 10.000 filas y 19 columnas, así que juntos forman exactamente las 40.000 filas que menciona el examen. No encontré valores nulos ni filas completamente duplicadas en ninguno de los lotes, ni tampoco al consolidarlos. 

Las 19 columnas son:

```
</> Texto sin formato
nombre_organizacion
sector_organizacion
pais_organizacion
tamano_empleados_organizacion
codigo_brecha
fecha_ocurrencia
fecha_deteccion
vector_ataque
severidad_incidente
registros_afectados_total
costo_estimado_total
codigo_usuario
pseudonimo_usuario
email_hash_usuario
pais_residencia_usuario
fecha_registro_usuario
tipo_dato_expuesto
categoria_sensibilidad_dato
fecha_notificacion_usuario
```

Por tipos aparentes, hay campos de texto/código, enteros, un decimal monetario y varias fechas. En concreto, `tamano_empleados_organizacion` y `registros_afectados_total` son enteros; `costo_estimado_total` es decimal; y las fechas vienen como texto en formato ISO `YYYY-MM-DD`, por lo que en PostgreSQL luego convendría tiparlas como `date` ↓

4. Normalización de la estructura

A partir de la sábana consolidada se inició la normalización hasta separar las entidades y la tabla puente requeridas.

ahora procedamos a normalizar el la estructura

Procesó durante 27s

Analizado >

Analizado

Analizado

Sí bro. Con la inspección lista, ahora sí podemos **normalizar formalmente la sábana**. Y lo bueno es que los datos reales nos llevan exactamente a las **5 tablas que anticipa el examen**. 

1NF — Primera Forma Normal

La sábana ya cumple 1NF porque cada columna contiene valores atómicos: una organización, una fecha, un vector de ataque, un tipo de dato, etc. No tenemos listas dentro de una celda ni grupos repetitivos.

La clave que identifica de manera única cada fila es:

```
</> Texto sin formato
(codigo_brecha, codigo_usuario, tipo_dato_expuesto)
```

Lo comprobamos sobre las 40.000 filas: esa combinación no se repite.

Entonces conceptualmente tenemos:

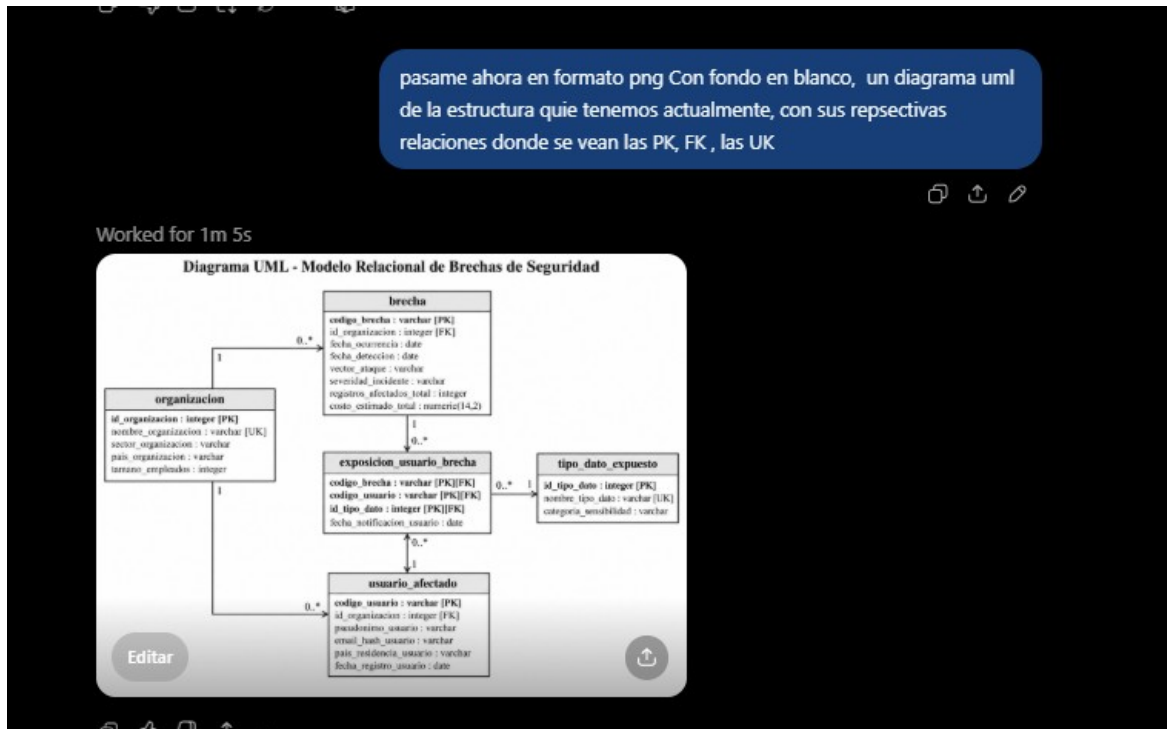
```
</> Texto sin formato
fecha_ocurrencia,
fecha_deteccion,
vector_ataque,
severidad_incidente,
registros_afectados_total,
costo_estimado_total,

codigo_usuario,
pseudonimo_usuario,
email_hash_usuario,
pais_residencia_usuario,
fecha_registro_usuario,

tipo_dato_expuesto,
extremidad, consistencia dato
```

5. Diagrama relacional

Se generó una primera versión del diagrama con las cinco tablas, sus claves PK, FK y UK, y las relaciones entre ellas.



Cabe aclarar que no me gusto como lo hizo, entonces lo decidí hacer a mano

6. Usuario, esquema y privilegios

Se definió la creación del usuario, el esquema brechas_seguridad y los privilegios mínimos necesarios para trabajar en PostgreSQL.

bueno bro empecemos con el bloque 1, creacion de usuario,esquema y asignacion de privilegios minimos

De una bro. Para el Bloque 1 yo lo dejaría simple y limpio: un usuario exclusivo para el examen, un esquema propio y únicamente los privilegios necesarios. El enunciado pide explícitamente creación del usuario/esquema y asignación de privilegios mínimos. 

Ejecuta este bloque conectado en DBeaver como `postgres` :

```
</> SQL
create role usuario_examen02
with
    login
    password 'Examen02_2026';

grant connect on database examen02
to usuario_examen02;

create schema brechas_seguridad
authorization usuario_examen02;

alter role usuario_examen02
set search_path to brechas_seguridad, public;
```

La idea es esta: `usuario_examen02` puede iniciar sesión y conectarse únicamente porque le damos `CONNECT` sobre `examen02` ; además, es propietario de `brechas_seguridad` , por lo que puede crear y administrar los objetos de su propio esquema sin necesidad de darle permisos elevados como `SUPERUSER` , `CREATEDB` o `CREATEROLE` .

Después podemos verificarlo con:

```
</> SQL
select
    rolname,
    rolsuper,
    rolcreatedb,
    rolcreaterole,
    rolcanlogin
from pg_roles
where rolname = 'usuario_examen02';
```

7. Creación de las tablas normalizadas

Se construyeron las tablas del modelo respetando los nombres del diagrama y definiendo sus claves primarias y foráneas.

siguiente, bloque de las tablas, en base porfa al diagrama relacional que te mande con los mismos nombres, con sus pk y fk

De una bro. Basándome en el diagrama relacional que me mandaste y manteniendo los mismos nombres de tablas y campos, este sería el bloque de creación de tablas con sus PK y FK.

```
</> SQL
create table organization (
  id_organizacion integer generated always as identity,
  nombre_organizacion varchar(150) not null,
  sector_organizacion varchar(100) not null,
  pais_organizacion varchar(100) not null,
  tamaño_empleados integer not null,

  constraint pk_organization
    primary key (id_organizacion),

  constraint uk_organization_nombre
    unique (nombre_organizacion)
);
```

```
</> SQL
create table brecha (
  codigo_brecha varchar(50) not null,
  id_organizacion integer not null,
  fecha_ocurrencia date not null,
  fecha_deteccion date not null,
  vector_etapa varchar(100) not null,
  severidad_incidente varchar(50) not null,
  registros_afectados_total integer not null,
  costo_estimado_total numeric(14,2) not null,

  constraint pk_brecha
    primary key (codigo_brecha),

  constraint fk_brecha_organizacion
    foreign key (id_organizacion)
      references organization (id_organizacion)
);
```

8. Creación de la tabla staging

Se creó una tabla staging con las 19 columnas originales para recibir los cuatro CSV antes de distribuir los datos normalizados.

empecemos bloque 3, vamos a crear una tabla staging para cargar los 4 csv,

De una bro. Para la staging, mi recomendación es que las 19 columnas sean `text`. La función de esta tabla es recibir los CSV **sin rechazar registros por tipos**; cuando pasemos a las tablas normalizadas hacemos las conversiones a `date`, `integer`, `numeric`, etc.

Bloque 3 — Tabla staging

Ejecutándolo como `samuel`, con el `search_path` que ya configuramos:

```
SQL

create table staging_brechas (
  nombre_organizacion text,
  sector_organizacion text,
  pais_organizacion text,
  tamano_empleados_organizacion text,

  codigo_brecha text,
  fecha_ocurrencia text,
  fecha_deteccion text,
  vector_ataque text,
  severidad_incidente text,
  registros_afectados_total text,
  costo_estimado_total text,

  codigo_usuario text,
  pseudonimo_usuario text,
  email_hash_usuario text,
  pais_residencia_usuario text,
  fecha_registro_usuario text,

  tipo_dato_expuesto text,
  capacidad_compartibilidad_dato text
```

No le ponemos PK, FK, UK, `NOT NULL` ni `CHECK` porque esta tabla no representa nuestro modelo relacional; es solamente una zona temporal de aterrizaje para los archivos originales.

El flujo de este bloque ahora es:

```
Texto sin formato

lote_1.csv ↘
lote_2.csv ↘
lote_3.csv → staging_brechas
lote_4.csv ↘
```

9. Carga de datos en las tablas finales

Se cargaron los datos desde staging hacia las tablas normalizadas y se verificaron los conteos esperados de cada entidad.

listo empecemos a cargar estos datos tabla por tabla

De una bro. Vamos tabla por tabla y no avanzamos hasta validar cada una.

Empezamos con `organizacion`. Como `staging_brechas` está en `public`, la leemos desde ahí y cargamos únicamente los valores distintos:

```
</> SQL
insert into brechas_seguridad.organizacion (
  nombre_organizacion,
  sector_organizacion,
  pais_organizacion,
  tamano_empleados
)
select distinct
  nombre_organizacion,
  sector_organizacion,
  pais_organizacion,
  tamano_empleados_organizacion::integer
from public.staging_brechas;
```

Ahora validamos:

```
</> SQL
select count(*)
from brechas_seguridad.organizacion;
```

Nos debe dar:

```
</> Texto sin formato
60
```

Y revisamos algunos registros:

```
</> SQL
select *
from brechas_seguridad.organizacion
order by id_organizacion
limit 10;
```

¿Por qué funciona? Porque en staging la organización aparece repetida muchas veces, pero sus atributos son consistentes. `distinct` nos deja solamente las 60 organizaciones únicas, y PostgreSQL genera automáticamente `id_organizacion` por el `identity`.

10. Validación y consolidación del modelo

Se revisaron restricciones, duplicados, nulos, integridad referencial y calidad de datos antes de cerrar la Etapa 2.

antes de pasar de etapa quiero, que corrijas el tema del esquema, valides el tema de integridad/calidad, evitar duplicados, nulos etnc, y consolidar el .sql definitivo

Procesó durante 1m 9s

Análisis en pausa ▾

</> Python

```
import pandas as pd, os, glob, json, numpy as np
files=glob.glob('/mnt/data/sabana_brechas_seguridad_lote_*.csv')
files
```

Analizado ▾

</> Python

```
from pathlib import Path

sql = r'''-- Examen 02 - Topicos Avanzados de Base de Datos
-- Samuel Parra Cadavid
-- ID SIGAA: 000551191
-- PostgreSQL 18.x
-- Base de datos: examen02
-- Esquema: brechas_seguridad

-- Este script esta organizado por bloques.
-- La carga de los cuatro CSV a staging_brechas se realiza desde DBeaver
-- despues de crear la tabla staging y antes de ejecutar el bloque de carga normalizada.

-- Bloque 1. Usuario, esquema y privilegios

create role samuel
with
    login
    password 'Examen123';

grant connect on database examen02
to samuel;

create schema brechas_seguridad
authorization samuel;

alter role samuel
set search_path to brechas_seguridad, public;

-- Bloque 2. Tablas normalizadas

create table brechas_seguridad.organizacion (
    id_organizacion integer generated always as identity,
    nombre_organizacion varchar(150) not null,
    sector_organizacion varchar(100) not null,
    pais_organizacion varchar(100) not null,
    tamano_empleados integer not null,

    constraint pk_organizacion
```

11. Consulta base de la Etapa 3

Se construyó la consulta de brechas críticas del último año y se preparó el análisis del plan de ejecución antes de optimizar.

Listo bro, pasamos a la Etapa 3: brechas críticas del último año con mayor impacto en usuarios.

El examen pide identificar, para el último año, las brechas de severidad Alta o Crítica que hayan expuesto al menos un dato con sensibilidad Crítica, mostrando organización, código de brecha, fecha de detección, vector de ataque y cantidad de usuarios distintos afectados; además debe ordenarse de mayor a menor por usuarios afectados.  `tsdb_202520_examen02_performanc...`

Como siempre, vamos por fases. Primero hacemos la consulta funcional sin optimizar, comprobamos que devuelva lo correcto y guardamos su plan de ejecución base. Después recién diseñamos índices y comparamos.

La lógica de la consulta es:

`</>` Texto sin formato

- brecha
- filtrar último año
- filtrar severidad Alta/Critica
- unir con exposicion_usuario_brecha
- unir con tipo_datos_expuesto
- exigir sensibilidad Crítica
- unir con organizacion
- contar usuarios distintos por brecha
- ordenar descendente

Primer SQL base:

`</>` SQL

```
select
  o.nombre_organizacion,
  b.codigo_brecha,
  b.fecha_deteccion,
  b.vector_ataque,
  count(distinct e.codigo_usuario) as usuarios_afectados
from brechas_seguridad.brecha b
join brechas_seguridad.organizacion o
  on o.id_organizacion = b.id_organizacion
join brechas_seguridad.exposicion_usuario_brecha e
  on e.codigo_brecha = b.codigo_brecha
join brechas_seguridad.tipo_datos_expuesto t
  on t.codigo_datos_expuesto = b.codigo_datos_expuesto
```

12. Optimización de la Etapa 3

Se analizó EXPLAIN (ANALYZE, BUFFERS) y se propusieron índices para reducir el costo de acceso y mejorar el tiempo de ejecución.

y luego baja correctamente a 15, 14, 13, etc.

Ahora viene la parte importante del examen: **performance tuning**. Antes de crear ningún índice, necesitamos capturar el plan base.

Ejecuta exactamente:

```
SQL

b.vector_ataque,
count(distinct e.codigo_usuario) as usuarios_afectados
from brechas_seguridad.brecha b
join brechas_seguridad.organizacion o
on o.id_organizacion = b.id_organizacion
join brechas_seguridad.exposicion_usuario_brecha e
on e.codigo_brecha = b.codigo_brecha
join brechas_seguridad.tipo_datos_exposicion t
on t.id_tipo_datos = e.id_tipo_datos
where b.fecha_deteccion >= (
select max(fecha_deteccion) - interval '1 year'
from brechas_seguridad.brecha
)
and b.severidad_incidente in ('Alta', 'Critica')
and t.categoria_sensibilidad = 'Critica'
group by
o.nombre_organizacion,
b.codigo_brecha,
b.fecha_deteccion,
b.vector_ataque
order by usuarios_afectados desc;
```

Ahora no crees índices todavía. Mándame el resultado completo del **EXPLAIN (ANALYZE, BUFFERS)** y lo leemos juntos: **Seq Scan**, **Hash Join**, **Sort**, costos, filas y tiempo. A partir de ese plan decidimos qué índice realmente tiene sentido.

13. Consulta y optimización de la Etapa 4

Se desarrolló la consulta del historial de exposición por usuario y se planteó un índice para comparar el rendimiento antes y después.

Ayúdame a resolver la Etapa 4 del examen en PostgreSQL. Necesito consultar el historial de exposición de un usuario por su código, mostrando brecha, tipo de dato expuesto, sensibilidad, organización y fecha de notificación, ordenado cronológicamente. Primero dame la consulta funcional y luego ayúdame a analizar EXPLAIN (ANALYZE, BUFFERS) para proponer un índice justificado y comparar el rendimiento antes y después.

Claro bro. Para la Etapa 4 puedes partir con una consulta funcional así:

```
</> SQL
select
  u.codigo_usuario,
  b.codigo_brecha,
  t.nombre_tipo_dato,
  t.categoria_sensibilidad,
  o.nombre_organizacion,
  e.fecha_notificacion_usuario
from brechas_seguridad.usuario_afectado u
join brechas_seguridad.exposicion_usuario_brecha e
  on e.codigo_usuario = u.codigo_usuario
join brechas_seguridad.brecha b
  on b.codigo_brecha = e.codigo_brecha
join brechas_seguridad.tipo_dato_expuesto t
  on t.id_tipo_dato = e.id_tipo_dato
join brechas_seguridad.organizacion o
  on o.id_organizacion = b.id_organizacion
where u.codigo_usuario = 'US-002823'
order by e.fecha_notificacion_usuario;
```

Luego ejecuta la misma consulta con:

```
</> SQL
explain (analyze, buffers)
...
```

Si el plan muestra un `Seq Scan` sobre `exposicion_usuario_brecha` filtrando por `codigo_usuario`, sería un buen candidato para optimizar porque esa tabla tiene muchos mas registros que las demas.

Un índice justificado sería: